

TRAVAUX PRATIQUES · STAGIAIRE

DOCKER SWARM EN PRATIQUE

Transformer plusieurs machines en un cluster, puis y déployer des services. Initialisation du Swarm, service web répliqué, réseau overlay et déploiement d'une application WordPress avec sa base de données.

~60

MINUTES

1

CLUSTER

WordPress

+ BASE DE DONNÉES

8

PARTIES



SPARKS
FORMATION

TP stagiaire · Docker Swarm
Session inter · du 06/07/2026 au 08/07/2026
Docker Engine 29.x · Compose v2 · Swarm

TP — À la découverte de Docker Swarm

Dans ce TP, vous allez transformer plusieurs machines Docker en un cluster, puis y déployer des services. Vous commencerez par un service web simple pour comprendre les mécanismes (répliques, répartition, auto-réparation), puis vous déploierez une application complète WordPress avec sa base de données.

Prérequis : au moins deux machines Linux avec Docker installé (une servira de *manager*, les autres de *workers*), qui peuvent communiquer entre elles par leur adresse IP. Les ports suivants doivent être ouverts entre les machines : `2377/tcp`, `7946/tcp+udp` et `4789/udp`.

Durée : environ 60 minutes.

Dans tout ce document, notez l'adresse IP de votre manager : vous en aurez besoin plusieurs fois. On l'écrira `IP_MANAGER`.

Partie 1 — Initialiser le cluster

Toutes les commandes de cette partie se lancent **sur la machine manager**. Initialisez le Swarm en indiquant l'adresse par laquelle les autres nœuds vous joindront :

```
docker swarm init --advertise-addr IP_MANAGER
```

La commande affiche une instruction `docker swarm join --token ...` : **copiez-la**, elle servira à la Partie 2. Vérifiez que votre nœud est actif :

```
docker node ls
```

Vous voyez une ligne, votre manager, avec le statut `Ready` et le rôle `Leader`.

Si vous perdez la commande de jointure, vous pouvez la réafficher à tout moment :

```
docker swarm join-token worker
```

Partie 2 — Ajouter les workers

Sur chaque machine worker, collez la commande de jointure copiée à la Partie 1. Elle ressemble à ceci :

```
docker swarm join --token SWMTKN-1-xxxxxxx IP_MANAGER:2377
```

Chaque worker répond `This node joined a swarm as a worker.`

Revenez **sur le manager** et listez les nœuds :

```
docker node ls
```

Vous voyez maintenant toutes vos machines, toutes en `Ready`. Le cluster est en place.

Partie 3 — Déployer un premier service

Un **service** décrit ce que le cluster doit maintenir en fonctionnement. Toutes les commandes de gestion se lancent **sur le manager**. Créez un service web en 3 exemplaires (répliques) :

```
docker service create --name web --replicas 3 -p 8081:80 nginx:1.30-alpine
```

Observez ce que le cluster a fait :

```
docker service ls          # le service web, 3/3 répliques
docker service ps web      # sur QUELS nœuds tournent les répliques
```

Testez la répartition de charge. Le port 8081 répond sur **n'importe quel nœud** du cluster, même ceux qui n'hébergent aucune réplique :

```
curl localhost:8081        # sur le manager
curl IP_D_UN_WORKER:8081   # depuis une autre machine : ça répond aussi
```

Augmentez le nombre de répliques, puis vérifiez :

```
docker service scale web=5
docker service ps web      # 5 répliques réparties sur les nœuds
```

Provoquez une panne pour voir l'auto-réparation. Sur un nœud qui héberge une réplique, supprimez son conteneur à la main :

```
docker ps                  # repérez l'ID d'un conteneur "web"
docker rm -f ID_DU_CONTENEUR
```

Puis, sur le manager :

```
docker service ps web      # le cluster a déjà recréé une réplique à la place
```

Le cluster maintient toujours l'état demandé : c'est le principe de l'orchestration. Supprimez ce service avant la suite :

```
docker service rm web
```

Partie 4 — Déployer WordPress avec sa base de données

Pour une application de plusieurs services, on décrit tout dans un fichier, puis on le déploie en une commande. **Sur le manager**, créez le fichier `wordpress-stack.yml` :

```
nano wordpress-stack.yml
```

Saisissez le contenu suivant :

```
services:
  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: rootpass
```

```

    MARIADB_DATABASE: wordpress
    MARIADB_USER: wordpress
    MARIADB_PASSWORD: wordpress
  volumes:
    - db_data:/var/lib/mysql
  networks:
    - wpnet
  deploy:
    replicas: 1
    placement:
      constraints:
        - node.role == manager

wordpress:
  image: wordpress:6
  ports:
    - "8080:80"
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
    WORDPRESS_DB_NAME: wordpress
  networks:
    - wpnet
  deploy:
    replicas: 2

networks:
  wpnet:
    driver: overlay

volumes:
  db_data:

```

Déployez la stack (le nom `wordpress` est celui de votre application) :

```
docker stack deploy -c wordpress-stack.yml wordpress
```

Suivez le démarrage des services :

```
docker stack services wordpress
```

Attendez que la colonne `REPLICAS` affiche `1/1` pour `db` et `2/2` pour `wordpress`. Le premier démarrage peut prendre une minute, le temps de tirer les images sur les nœuds.

Partie 5 — Accéder à WordPress

Grâce au réseau overlay et à la répartition de charge, WordPress répond sur le port 8080 de **n'importe quel nœud**. Ouvrez dans votre navigateur :

```
http://IP_MANAGER:8080
```

L'assistant d'installation de WordPress s'affiche. Choisissez la langue, renseignez le titre du site, un identifiant et un mot de passe d'administration, puis validez. Connectez-vous : votre site fonctionne, servi par le cluster.

Rechargez la page en utilisant l'adresse d'un autre nœud (`http://IP_D_UN_WORKER:8080`) : le site répond de la même façon. La base de données, elle, tourne sur le manager (où se trouve son volume), et les deux répliques WordPress s'y connectent par le nom de service `db`.

Partie 6 — Observer et faire évoluer la stack

Les services d'une stack sont préfixés par son nom : `wordpress_db` et `wordpress_wordpress`.

```
docker stack ps wordpress          # toutes les tâches et leurs nœuds
docker service ps wordpress_wordpress # les répliques WordPress
docker service logs wordpress_wordpress # les journaux
```

Augmentez le nombre de répliques WordPress :

```
docker service scale wordpress_wordpress=3
docker service ps wordpress_wordpress      # 3 répliques désormais
```

Mettez à jour l'image sans interrompre le service. Le cluster remplace les répliques une par une :

```
docker service update --image wordpress:6.6 wordpress_wordpress
docker service ps wordpress_wordpress      # les tâches basculent progressivement
```

Nettoyage

Supprimez la stack déployée :

```
docker stack rm wordpress
```

Si vous souhaitez aussi dissoudre le cluster, faites quitter chaque machine. Sur chaque **worker** :

```
docker swarm leave
```

Puis sur le **manager** :

```
docker swarm leave --force
```